

CENG 467 Natural Language Understanding and Generation

Take-Home Midterm Report

Student Name: **Seda Güven**
Student ID: **300201064**

07.05.2026

Abstract

This report presents five experiments on core topics in natural language understanding and generation: sentiment classification, named entity recognition, text summarization, machine translation, and language modeling. Each task compares different modeling approaches, ranging from classical feature-based methods to neural sequence models and pretrained Transformer models. The experiments are conducted on standard benchmark datasets, including IMDb, CoNLL-2003, CNN/DailyMail, Multi30k, and WikiText-2. Performance is evaluated using task-appropriate metrics such as Accuracy, Macro-F1, Precision, Recall, F1-score, ROUGE, BLEU, METEOR, ChrF, BERTScore, and perplexity. In addition to reporting quantitative results, the report includes error analysis and qualitative examples to examine model behavior, limitations, and trade-offs. Overall, the results show that pretrained Transformer-based models generally achieve the strongest performance, while classical and recurrent baselines remain useful for interpretability, controlled comparison, and understanding the effect of representation choices.

Contents

1	Question 1: Representation Learning in Text Classification	3
1.1	Problem Formulation and Dataset	3
1.2	Preprocessing and Tokenization	3
1.3	Models and Experimental Setup	4
1.4	Evaluation Results	4
1.5	Misclassification Analysis	5
1.6	Discussion	5
2	Question 2: Named Entity Recognition	6
2.1	Problem Formulation and Dataset	6
2.2	Preprocessing and Label Alignment	6
2.3	Models and Experimental Setup	6
2.4	Evaluation Results	7
2.5	Error Analysis	7
2.6	Discussion	8
3	Question 3: Text Summarization	9
3.1	Problem Formulation and Dataset	9
3.2	Methods and Experimental Setup	9
3.3	Evaluation Results	10
3.4	Qualitative Analysis	10
3.5	Discussion	11

4	Question 4: Machine Translation	12
4.1	Problem Formulation and Dataset	12
4.2	Preprocessing and Models	12
4.3	Evaluation Results	13
4.4	Qualitative Analysis	13
4.5	Discussion	14
5	Question 5: Language Modeling	15
5.1	Problem Formulation and Dataset	15
5.2	Preprocessing and Models	15
5.3	Evaluation Results	16
5.4	Text Generation	16
5.5	Discussion	17
	Repository Link	17

1 Question 1: Representation Learning in Text Classification

1.1 Problem Formulation and Dataset

This task was formulated as binary sentiment classification on the IMDb movie review dataset. Given a review text, the model predicts whether the sentiment is negative or positive. The main goal was to compare three representation types: sparse lexical features, dense sequential representations, and contextual transformer representations.

IMDb was selected because its reviews are longer than sentence-level sentiment datasets and often include mixed opinions, plot descriptions, delayed judgments, and genre-specific comments. This makes it useful for testing how different representations handle long and sometimes ambiguous sentiment signals.

The original IMDb training split contains 25,000 labeled reviews, and the official test split contains 25,000 reviews. The training split was divided into 20,000 training examples and 5,000 validation examples using stratified sampling with random seed 42. The official test set was only used after finalizing the models and preprocessing choices.

Table 1: IMDb dataset splits used in the experiments.

Split	Examples	Negative	Positive
Train	20,000	10,000	10,000
Validation	5,000	2,500	2,500
Test	25,000	12,500	12,500

1.2 Preprocessing and Tokenization

The same train, validation, and test splits were used for all models. Tokenization was model-specific because each representation requires a different input format.

For the TF-IDF baseline, text was lowercased and represented with word unigram and bigram features. The main configuration used a maximum vocabulary size of 30,000 and minimum document frequency of 2. Stopwords were not removed in the final setting because words such as “not”, “no”, and “never” are important for sentiment.

For the BiLSTM model, a word-level tokenizer was used. The vocabulary was built only from the training set to avoid data leakage. Reviews were truncated or padded to 256 tokens, and packed sequences were used so that padding tokens were ignored by the LSTM.

For DistilBERT, the pretrained `distilbert-base-uncased` WordPiece tokenizer was used. Reviews were truncated to 256 subword tokens and dynamically padded inside each batch.

A small preprocessing ablation was performed with TF-IDF + Logistic Regression. The best setting was full text without stopword removal.

Table 2: Preprocessing ablation using TF-IDF + Logistic Regression on the validation set.

Setting	Accuracy	Macro-F1
Full text, no stopword removal	0.8966	0.8966
Full text, English stopword removal	0.8930	0.8930
First 256 words only	0.8838	0.8838

Stopword removal slightly reduced performance, most likely because sentiment-bearing function words and negations were removed. Truncating reviews to the first 256 words caused a larger drop, showing that important sentiment information often appears later in IMDb reviews.

1.3 Models and Experimental Setup

Three models were implemented under the same data split and evaluation protocol.

TF-IDF + Logistic Regression was used as the sparse lexical baseline. It used unigram and bigram TF-IDF features with Logistic Regression. This model is easy to inspect because feature weights correspond to words or phrases.

Packed BiLSTM was used as the dense sequential model. It used randomly initialized word embeddings, a bidirectional LSTM, dropout, and a linear classifier. Packed sequences were used to reduce the effect of padding. Since the embeddings were learned only from IMDB supervision, this model had less external linguistic knowledge than DistilBERT.

DistilBERT was used as the contextual transformer model. The pretrained `distilbert-base-uncased` model was fine-tuned for binary sequence classification. Its WordPiece tokenizer and self-attention layers allow it to represent words in context.

Table 3: Main experimental settings for Question 1.

Component	Setting
Dataset	IMDb
Train / Validation / Test	20,000 / 5,000 / 25,000
Random seed	42
Metrics	Accuracy, Macro-F1
TF-IDF features	Unigrams + bigrams, max features = 30,000
Logistic Regression	<code>liblinear</code> , max iterations = 1000
BiLSTM	Embedding dim = 128, hidden dim = 128, dropout = 0.5
BiLSTM max length	256 tokens
DistilBERT model	<code>distilbert-base-uncased</code>
DistilBERT max length	256 subword tokens

1.4 Evaluation Results

Accuracy and Macro-F1 were used as the main metrics. Since the dataset splits are balanced, the two values are close, but Macro-F1 is still reported to check whether both classes are handled similarly.

Table 4: Validation results of the three representation approaches.

Model	Accuracy	Macro-F1
TF-IDF + Logistic Regression	0.8966	0.8966
Packed BiLSTM	0.8382	0.8377
DistilBERT	0.9138	0.9138

Table 5: Final test results on the official IMDb test set.

Model	Accuracy	Macro-F1
TF-IDF + Logistic Regression	0.8893	0.8893
Packed BiLSTM	0.8162	0.8153
DistilBERT	0.9120	0.9120

DistilBERT achieved the best performance on both validation and test sets. TF-IDF was also strong and remained close to DistilBERT, which shows that sparse lexical features are

highly competitive for IMDb sentiment classification. The BiLSTM performed worse than both models, suggesting that a neural sequence model trained from scratch is not necessarily better than a strong sparse baseline.

1.5 Misclassification Analysis

Five high-confidence validation errors were inspected for each model. Instead of listing full reviews, Table 6 summarizes the main error patterns.

Table 6: Summary of common high-confidence validation errors.

Model	Typical Error Pattern	Example Interpretation
TF-IDF + Logistic Regression	Misleading lexical cues	Positive reviews with words such as “bad”, “terrible”, or “weak” were sometimes predicted as negative.
Packed BiLSTM	Sentiment shift and truncation	Reviews that began negatively but later became positive were often classified incorrectly.
DistilBERT	Genre-specific or ironic sentiment	Reviews describing a movie as bad but enjoyable, especially cult or horror films, still caused errors.
All models	Mixed sentiment and label ambiguity	Some reviews contained both praise and criticism, or the IMDb rating label did not clearly match the review tone.

The error analysis shows that the remaining mistakes are not only caused by missing sentiment words. Many errors require understanding the final stance of the reviewer, genre expectations, irony, or contrastive structure. For example, low-budget horror reviews may use negative words while still giving a positive rating because the viewer enjoyed the film as a genre example. This was difficult even for DistilBERT.

1.6 Discussion

The results show a clear difference between the three representation types. TF-IDF is simple, fast, and interpretable. It works well because IMDb reviews often contain explicit sentiment words and phrases. However, it struggles when surface polarity conflicts with the actual opinion.

The BiLSTM captures word order better than TF-IDF, but it underperformed in this setup. The main reasons are that it used randomly initialized embeddings, only supervised IMDb data, and truncated input. It also showed signs of overfitting after the best validation epoch.

DistilBERT gave the best overall performance because it uses pretrained contextual representations. It handles local context and negation better than the other two models. Still, its errors show that contextual embeddings do not fully solve pragmatic cases such as irony, mixed sentiment, or genre-specific enjoyment.

Overall, the strongest model was DistilBERT, the most interpretable model was TF-IDF + Logistic Regression, and the weakest model in this experiment was the BiLSTM trained from scratch. The main conclusion is that representation quality matters: pretrained contextual representations generalize best, while sparse lexical features remain a strong and efficient baseline.

2 Question 2: Named Entity Recognition

2.1 Problem Formulation and Dataset

Named Entity Recognition (NER) was treated as a sequence labeling task. Given a token sequence, the model predicts one label for each token. The aim is to identify named entities and assign them to one of the entity types.

The CoNLL-2003 dataset was used for this experiment. It contains English news sentences annotated with named entity labels. The entity categories are person (PER), organization (ORG), location (LOC), and miscellaneous (MISC). The annotations follow the BIO scheme, where B-X marks the beginning of an entity of type X, I-X marks a continuation token inside the same entity, and O marks tokens outside named entities.

The complete NER label set is:

$$\{O, B\text{-}PER, I\text{-}PER, B\text{-}ORG, I\text{-}ORG, B\text{-}LOC, I\text{-}LOC, B\text{-}MISC, I\text{-}MISC\}.$$

Table 7: CoNLL-2003 dataset statistics.

Split	Sentences	Tokens
Train	14,041	203,621
Validation	3,250	51,362
Test	3,453	46,435

Most tokens in the dataset are labeled as O, which is expected for NER because named entities are sparse compared with ordinary words. Therefore, token-level accuracy was not used as the main evaluation measure. Entity-level Precision, Recall, and F1-score were used instead.

2.2 Preprocessing and Label Alignment

For the CRF model, the original word-level tokens from CoNLL-2003 were used directly. Each token already has one BIO label, so no additional label alignment was needed.

For the transformer model, label alignment was required because DistilBERT uses subword tokenization. A word can be split into multiple subword pieces. In this implementation, the original word label was assigned only to the first subword. The remaining subwords and special tokens such as [CLS] and [SEP] were assigned the ignore index -100. This prevents artificial subword positions from affecting the loss or evaluation.

2.3 Models and Experimental Setup

Two models were implemented and compared: a classical CRF model and a transformer-based DistilBERT token classification model.

CRF was used as the classical sequence labeling baseline. It used manually designed token-level features such as lowercase word form, prefixes, suffixes, capitalization, digit information, hyphen/dot indicators, and neighboring token features. The CRF also learns transition patterns between BIO tags, which is useful for avoiding invalid or unlikely label sequences.

DistilBERT was used as the contextual model. The model was based on `distilbert-base-cased`, since capitalization is important in NER. A token classification head was added and fine-tuned on the aligned BIO labels.

Table 8: Experimental setup for Question 2.

Component	Setting
Dataset	CoNLL-2003
Splits	Official train / validation / test splits
Random seed	42
Classical model	CRF
Transformer model	distilbert-base-cased
Evaluation metrics	Entity-level Precision, Recall, F1-score
CRF optimizer	L-BFGS
CRF regularization	$c_1 = 0.1, c_2 = 0.1$
CRF max iterations	100
DistilBERT max sequence length	128
DistilBERT batch size	16
DistilBERT learning rate	2×10^{-5}
DistilBERT epochs	3
Weight decay	0.01
Environment	Google Colab with CUDA GPU

2.4 Evaluation Results

The models were evaluated using entity-level Precision, Recall, and F1-score. This evaluation is stricter than token-level accuracy because an entity is counted as correct only when both the span and entity type are correct.

Table 9: NER performance on validation and test sets.

Model	Split	Precision	Recall	F1-score
CRF	Validation	0.8969	0.8650	0.8807
CRF	Test	0.8197	0.7923	0.8058
DistilBERT	Validation	0.9309	0.9380	0.9345
DistilBERT	Test	0.8805	0.8995	0.8899

DistilBERT outperformed the CRF model on both validation and test sets. On the test set, DistilBERT reached an F1-score of 0.8899, while CRF reached 0.8058. The improvement is especially clear in recall, which suggests that contextual embeddings helped the model detect more gold entities.

2.5 Error Analysis

Validation errors were grouped into four categories: entity type confusion, boundary error, missed entity, and spurious entity. Entity type confusion means the span is detected but assigned the wrong entity type. Boundary error means the predicted entity overlaps with the gold entity but has an incorrect span. Missed entity means a gold entity is not predicted, and spurious entity means the model predicts an entity where no gold entity exists.

Table 10: Validation error type counts for CRF and DistilBERT.

Error Type	CRF Count	DistilBERT Count
Entity type confusion	364	183
Boundary error	174	126
Missed entity	264	59
Spurious entity	50	66

The CRF made more entity type confusion errors and missed entity errors. For example, some nationality or demonym-like expressions were confused with locations or miscellaneous entities. This is expected because the CRF depends heavily on local surface features.

DistilBERT reduced most error types, especially missed entities. This shows the advantage of contextual token representations. However, it still made boundary mistakes on multi-token entities and type confusion errors when the entity surface form was ambiguous. For instance, team or organization names can sometimes look similar to person names without broader context.

Spurious entities were slightly higher for DistilBERT. Some of these errors came from capitalized tokens or news-style formatting, where the model predicted an entity even though the gold label was 0. This shows that contextual models are stronger overall but still not perfect.

2.6 Discussion

The results show that contextual embeddings are important for NER. The CRF model benefits from explicit BIO transition modeling and interpretable hand-crafted features, but its context is limited. It can learn that certain label transitions are likely, but it cannot deeply represent the meaning of a token in the full sentence.

DistilBERT performed better because each token representation is computed using the surrounding sentence context. This helps when the entity type cannot be decided from the word alone. The large reduction in missed entities also suggests that contextual embeddings improve entity coverage.

Still, the remaining errors show that NER is not solved completely. Both models struggled with exact boundaries in multi-token entities and with ambiguous entity types. Overall, DistilBERT provided stronger performance and better generalization, while the CRF remained a useful classical baseline for showing the value of sequence-level label dependencies.

3 Question 3: Text Summarization

3.1 Problem Formulation and Dataset

This question compares extractive and abstractive summarization. The task is to generate a shorter summary from a longer news article while preserving the main information.

The CNN/DailyMail dataset was used. Each example contains a news article, a human-written reference summary in the `highlights` field, and an ID. Version 3.0.0 of the dataset was used. Since abstractive generation and BERTScore evaluation are computationally more expensive, the experiment was conducted on a reproducible subset of 20 examples from the test split. The test split was shuffled with random seed 42, and the first 20 examples were selected.

Table 11: Statistics of the selected CNN/DailyMail test subset.

Statistic	Article Words	Summary Words	Article Sentences	Summary Sentences
Mean	739.15	51.45	37.85	3.50
Std. Dev.	352.37	22.61	21.28	1.36
Min	294.00	26.00	14.00	2.00
Max	1430.00	107.00	88.00	7.00

The selected articles are much longer than their reference summaries, so the task requires strong compression. On average, articles contain 739.15 words, while reference summaries contain 51.45 words.

3.2 Methods and Experimental Setup

Two summarization methods were compared.

TextRank was implemented as the extractive method. Articles were split into sentences using NLTK. Very short sentences were removed because they often corresponded to captions or fragments. The remaining sentences were represented with TF-IDF vectors, pairwise cosine similarity was computed, and PageRank was applied on the sentence similarity graph. The top 3 ranked sentences were selected and reordered according to their original positions in the article.

BART was used as the abstractive method. The pretrained `facebook/bart-large-cnn` model was used without further fine-tuning. Unlike TextRank, BART generates new text, so it can paraphrase and compress information instead of only copying source sentences.

Table 12: Experimental setup for Question 3.

Component	Setting
Dataset	CNN/DailyMail 3.0.0
Split	Test subset
Subset size	20 examples
Random seed	42
Extractive model	TextRank
Abstractive model	facebook/bart-large-cnn
TextRank sentence representation	TF-IDF
TextRank similarity	Cosine similarity
TextRank selected sentences	3
BART max input length	1024 tokens
BART min / max summary length	30 / 100 tokens
BART decoding	Beam search, 4 beams
Generation time	7.41 seconds total, 0.37 seconds/example

3.3 Evaluation Results

The generated summaries were evaluated with ROUGE-1, ROUGE-2, ROUGE-L, BLEU, METEOR, and BERTScore. ROUGE measures lexical overlap, BLEU focuses on n-gram precision, METEOR uses more flexible alignment, and BERTScore measures contextual semantic similarity.

Table 13: Automatic summarization results on the selected CNN/DailyMail subset.

Model	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR	BERTScore-P	BERTScore-R	BERTScore-F1
TextRank	0.3449	0.1393	0.2333	8.6269	0.3404	0.8669	0.8706	0.8686
BART	0.4229	0.1669	0.2756	12.2576	0.3825	0.8809	0.8798	0.8802

BART achieved higher scores on all metrics. This shows that its summaries were closer to the reference summaries both lexically and semantically. The improvement is expected because **facebook/bart-large-cnn** is pretrained and fine-tuned for CNN/DailyMail-style summarization.

3.4 Qualitative Analysis

Three examples were inspected manually to compare fluency, factual consistency, and information coverage. Instead of including full articles, Table 14 shows the main observations from the qualitative analysis.

Table 14: Qualitative comparison of TextRank and BART summaries.

Example	TextRank Behavior	BART Behavior	Main Observation
0	Selected the key sentence “We should legalize medical marijuana.”	Produced a fluent summary but attributed the article to “John Sutter” instead of Dr. Sanjay Gupta.	BART was fluent but made a factual attribution error.
1	Covered the child, Twitter account, gun video, and backlash, but included caption-like fragments such as “Baby-faced” and “Backlash.”	Produced a more compact summary, but focused more on offensive tweet content than the reference.	TextRank was faithful but less clean; BART was more readable but shifted emphasis.
2	Captured the Chris Christie town hall conflict but included less useful caption-like wording such as “Not being nice.”	Gave a more readable summary and included the ExxonMobil settlement context.	BART improved fluency, while TextRank copied noisy article fragments.

The examples show the main difference between the two approaches. TextRank is usually more directly faithful because it copies source sentences, but it may include noisy or poorly connected sentences. BART produces more readable summaries, but it can introduce factual mistakes or omit specific details from the reference.

3.5 Discussion

The automatic metrics favor BART. It achieved better ROUGE, BLEU, METEOR, and BERTScore values than TextRank. This indicates better overlap with reference summaries and stronger semantic similarity.

However, the qualitative analysis shows that higher metric scores do not guarantee factual correctness. In one example, BART produced an incorrect attribution. This is important for news summarization because names and sources must be accurate. TextRank avoids this type of generation error because it only extracts original sentences, but its summaries are less polished and sometimes include caption-like fragments from the article.

The trade-off is therefore clear. TextRank is simple, computationally light, and more directly faithful to the source wording. BART is more fluent and better at compression, but it is computationally heavier and can introduce factual inconsistencies. For this subset, BART was stronger according to the metrics, but manual inspection was still necessary to understand its limitations.

4 Question 4: Machine Translation

4.1 Problem Formulation and Dataset

This question compares a Seq2Seq model with attention and a pretrained Transformer model for English-to-German machine translation. The goal is to translate an English source sentence into German while preserving its meaning.

The Multi30k dataset was used. It contains short image-caption sentences and their translations. In this experiment, English sentences were used as the source language and German sentences as the target language. The dataset provides separate train, validation, and test splits. To keep the experiment manageable, a reproducible subset was used: 8,000 training examples, 1,000 validation examples, and 100 test examples. The random seed was fixed to 42.

Table 15: Sentence length statistics for the selected Multi30k subsets.

Split	Language	Mean	Std. Dev.	Min	Max
Train	English	13.07	4.11	4	40
Train	German	12.48	4.26	1	43
Validation	English	13.07	3.88	4	30
Validation	German	12.61	4.18	3	33
Test	English	13.48	4.31	7	29
Test	German	12.92	4.41	6	27

The sentences are short, but the dataset still contains many specific visual-description words such as objects, actions, clothing, and places. This makes rare-word handling important, especially for a small word-level Seq2Seq model.

4.2 Preprocessing and Models

For the custom Seq2Seq model, a simple word-level preprocessing pipeline was used. Sentences were lowercased, punctuation was separated, and separate English and German vocabularies were built. The special tokens `<pad>`, `<unk>`, `<sos>`, and `<eos>` were added. The vocabulary was built with `min_freq=1`, so every token appearing in the selected training subset was included.

The first model was a GRU-based Seq2Seq model with attention. The encoder was a bidirectional GRU, and the decoder was a GRU decoder with additive attention over the encoder outputs. The model generated German tokens one by one using greedy decoding. A simple repetition-control rule was added because the model sometimes repeated frequent words such as *und*.

The second model was the pretrained Transformer model `Helsinki-NLP/opus-mt-en-de`. It was not fine-tuned in this experiment and was used directly for inference. Unlike the custom Seq2Seq model, this model uses subword tokenization and was pretrained on large-scale translation data.

Table 16: Experimental setup for Question 4.

Component	Setting
Dataset	Multi30k
Task	English-to-German translation
Train / Validation / Test	8,000 / 1,000 / 100
Random seed	42
Seq2Seq encoder	Bidirectional GRU
Seq2Seq decoder	GRU with additive attention
Embedding size	128
Hidden size	256
Dropout	0.3
Seq2Seq parameters	11,366,598
Seq2Seq training epochs	10
Optimizer	Adam, learning rate 0.001
Teacher forcing ratio	0.5
Best validation loss	4.113
Seq2Seq training time	41.96 seconds
Transformer model	Helsinki-NLP/opus-mt-en-de
Transformer decoding	Beam search, 4 beams
Total translation time	10.26 seconds for 100 test examples

4.3 Evaluation Results

Translation quality was evaluated using BLEU, METEOR, ChrF, and BERTScore. BLEU measures exact n-gram precision with a brevity penalty. METEOR uses more flexible alignment. ChrF measures character n-gram similarity, which is useful for German because of morphology and compound-like word forms. BERTScore measures semantic similarity using contextual embeddings.

Table 17: Machine translation results on the Multi30k test subset.

Model	BLEU	METEOR	ChrF	BERTScore-P	BERTScore-R	BERTScore-F1
Seq2Seq + Attention	3.2905	0.4771	35.3786	0.7499	0.7370	0.7430
Pretrained Transformer	36.2735	0.6752	63.7679	0.8984	0.9031	0.9006

The pretrained Transformer clearly outperformed the Seq2Seq model on all metrics. The BLEU gap is especially large, showing that the Seq2Seq translations rarely matched the reference at the phrase level. ChrF is also much higher for the Transformer, indicating better character-level similarity to the German references. BERTScore confirms that the Transformer preserved meaning better as well.

4.4 Qualitative Analysis

Table 18 shows representative examples from the test subset. The examples were selected because they clearly show the difference between the small Seq2Seq model and the pretrained Transformer.

Table 18: Qualitative comparison of Seq2Seq and Transformer translations.

Source English	Reference German	Seq2Seq + Attention	Pretrained Transformer	Transformer
A boy at a gun range aims and shoots.	Ein Junge zielt und schießt auf einem Schießstand.	ein junge, ein paar und und, wie er.	Ein Junge auf einem Schießstand zielt und schießt.	
A man on a scaffold in front of a house is smiling and posing for the photographer.	Ein Mann auf einem Gerüst vor einem Haus lächelt und posiert für den Fotografen.	ein mann auf einem vor vor einem und und posiert für für.	Ein Mann auf einem Gerüst vor einem Haus lächelt und posiert für den Fotografen.	
Two people are sitting fishing on striped beach chairs in a body of water.	Zwei Personen sitzen auf gestreiften Liegestühlen und angeln in einem Gewässer.	zwei personen sitzen auf einem gestreiften und und auf einem gewässer.	Zwei Leute sitzen auf gestreiften Strandstühlen in einem Körper aus Wasser angeln.	

The Seq2Seq model learned some common German sentence patterns, such as *ein mann*, *ein junge*, and *zwei personen*. However, it often failed on specific content words and produced repetitions such as *und und*. This happened especially with rare or visually specific words such as “gun range”, “scaffold”, and “beach chairs”. Greedy decoding also made the output fragile because early mistakes affected the rest of the sentence.

The Transformer outputs were much more fluent and complete. It correctly translated “gun range” as *Schießstand* and “scaffold” as *Gerüst*. The third example also shows that the Transformer is not perfect: “body of water” was translated literally as *Körper aus Wasser*, which is unnatural in German. Still, the Transformer preserved much more meaning than the Seq2Seq model.

4.5 Discussion

The results show a clear advantage for the pretrained Transformer. It performed better according to all metrics and produced more fluent translations. This is mainly because it was pretrained on large translation corpora and uses subword tokenization, which helps with rare and morphologically complex words.

The Seq2Seq model with attention was useful as a baseline because it demonstrates encoder-decoder translation and attention. However, it was trained from scratch on only 8,000 sentence pairs and used word-level tokens. As a result, it struggled with rare words, repetition, and full sentence coherence. Attention helped the model condition on source tokens, but it was not enough to match the pretrained Transformer.

The metrics reflect different aspects of the results. BLEU strongly penalized the Seq2Seq model because its phrase-level matches were weak. ChrF gave it a higher relative score because some character-level similarity remained. METEOR and BERTScore were more tolerant and showed that the Seq2Seq model sometimes preserved partial meaning. Overall, the Transformer was stronger in both automatic evaluation and qualitative inspection, while the Seq2Seq model showed the limitations of training a small recurrent translation model from scratch.

5 Question 5: Language Modeling

5.1 Problem Formulation and Dataset

This question investigates language modeling and text generation. The task is next-token prediction: given a sequence of previous tokens, the model predicts the next token. Lower uncertainty in next-token prediction should correspond to lower perplexity.

The WikiText-2 dataset was used. It contains Wikipedia-style text and provides train, validation, and test splits. To keep training time manageable, reproducible subsets were selected: 8,000 non-empty training lines, 1,000 validation lines, and 1,000 test lines. The random seed was fixed to 42.

Table 19: Basic statistics of non-empty WikiText-2 lines.

Split	Non-empty Lines	Mean Tokens/Line	Max Tokens/Line
Train	23767	87.85	721
Validation	2461	88.52	434
Test	2891	84.87	483

WikiText-2 contains many named entities, dates, titles, and rare words. This makes it challenging for a word-level model trained from scratch.

5.2 Preprocessing and Models

For the LSTM model, text was lowercased, punctuation was separated, and a word-level vocabulary was built from the selected training subset. Three special tokens were used: `<pad>`, `<unk>`, and `<eos>`. Tokens appearing fewer than 3 times were mapped to `<unk>`. The final vocabulary size was 16,174.

The text was converted into a continuous token stream. Fixed-length sequences of 50 tokens were created. For each input sequence, the target sequence was shifted one position to the right.

Two models were compared:

LSTM language model: a two-layer LSTM trained from scratch on WikiText-2. It used word embeddings, dropout, and a linear output layer over the vocabulary.

GPT-2: a pretrained Transformer-based causal language model. It was not fine-tuned; it was used directly for perplexity evaluation and generation. Unlike the LSTM, GPT-2 uses subword tokenization and was pretrained on a much larger corpus.

Table 20: Experimental setup for Question 5.

Component	Setting
Dataset	WikiText-2
Train / Validation / Test subset	8,000 / 1,000 / 1,000 lines
Random seed	42
LSTM vocabulary size	16,174
LSTM sequence length	50 tokens
LSTM embedding size	256
LSTM hidden size	512
LSTM layers	2
Dropout	0.3
LSTM parameters	16,116,014
Optimizer	Adam, learning rate 0.001
Training epochs	8
Best validation loss	5.2674
Best validation perplexity	193.92
LSTM training time	14.06 seconds
Transformer model	<code>gpt2</code>

5.3 Evaluation Results

The models were evaluated using perplexity. Perplexity is the exponential of the average loss:

$$\text{Perplexity} = e^{\text{loss}}.$$

A lower perplexity means that the model assigns higher probability to the correct next tokens.

Table 21: Language modeling results on the WikiText-2 test subset.

Model	Test Loss	Test Perplexity
LSTM	5.1925	179.91
GPT-2	3.8837	48.60

GPT-2 achieved much lower perplexity than the LSTM model. This is expected because GPT-2 is pretrained on large-scale text and uses subword tokenization, while the LSTM was trained from scratch on a small subset with word-level tokens.

5.4 Text Generation

Both models were used to generate short samples from the same prompts. The LSTM used temperature sampling over its word-level vocabulary. GPT-2 used sampling with temperature, top-k sampling, and nucleus sampling.

Table 22: Generated text samples from the LSTM and GPT-2 language models.

Prompt	LSTM Generation	GPT-2 Generation
The history of	the history of the end of a <unk> . <unk> makes the american <unk> language <unk> <unk> “ and <unk> “ in a <unk> , who had been able to the opportunity of the <unk> .	The history of the United States is in many ways a tale of three separate empires. The first of these empires, the British Empire, came about through the efforts of King George III of England.
In the city	in the city .	In the city of New Brunswick, a police sergeant was shot to death after a suspected drug deal. The alleged victim, who was an officer in the police force’s narcotics unit, had been living in the town for several months.
The film was	the film was werneth with tellicherry ’ s test . because of the cinema , none of the british <unk> , infrastructure @-@ <unk> , and <unk> and <unk> .	The film was shot by director Andrew Dyson and co-writer Jonathan Koppel, who are both Jewish. It was released on April 10, 1989.

The LSTM samples show limited coherence. The model sometimes begins with a reasonable local phrase, but the text quickly becomes fragmented or repetitive. It also produces many <unk> tokens because rare words are not represented directly in the word-level vocabulary. In the second prompt, it generated only “in the city .”, which shows that it can choose short and safe continuations.

GPT-2 generated more fluent and longer continuations. Its outputs had clearer sentence structure and better local coherence. However, the generated content should not be treated as factual. For example, GPT-2 can produce plausible names, dates, or events that are not necessarily true.

5.5 Discussion

The perplexity and generation results show the same pattern. The LSTM learned basic local language patterns, but it struggled with rare words and longer coherence. Its word-level vocabulary caused unknown tokens, and training from scratch on 8,000 lines was not enough to produce strong generation quality.

GPT-2 performed better because it has three main advantages: large-scale pretraining, subword tokenization, and self-attention. Subword tokenization reduces the rare-word problem, and self-attention helps the model use wider context than the recurrent LSTM.

The LSTM is still useful for demonstrating probabilistic sequence modeling: it predicts the next token at each step and its quality can be measured directly with perplexity. However, for actual generation quality, the pretrained Transformer was clearly stronger. The main limitation of GPT-2 is different: its text is fluent, but fluency does not guarantee factual correctness.

Repository Link

GitHub repository link: https://github.com/guvenseda01/Natural_Language_Processing_Midterm